32.4 安全审计

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~290k tokens，输出 ~8k tokens（本节）

安全配置需要持续验证——即使系统有良好的默认值，用户的自定义配置仍可能引入安全漏洞。OpenClaw 内置了一套自动化安全审计系统，可以扫描配置、文件权限、渠道策略等多个维度，生成结构化的安全发现报告。本节分析这套系统的架构与实现。

32.4.1 审计系统（`src/security/audit.ts`）

审计报告结构

安全审计的输出是一个结构化的 `SecurityAuditReport`：

```
// src/security/audit.ts

type SecurityAuditSeverity = "info" | "warn" | "critical";

type SecurityAuditFinding = {
  checkId: string;         // 检查项 ID（如 "gateway.bind_no_auth"）
  severity: SecurityAuditSeverity;
  title: string;           // 简短标题
  detail: string;          // 详细描述
  remediation?: string;    // 修复建议
};

type SecurityAuditReport = {
  ts: number;                          // 审计时间戳
  summary: { critical, warn, info };   // 各级别计数
  findings: SecurityAuditFinding[];    // 所有发现
  deep?: { gateway?: { ... } };        // 深度探测结果
};
```

审计流水线

`runSecurityAudit` 是审计的入口函数，它按类别顺序收集发现：

```
// src/security/audit.ts（简化）

export async function runSecurityAudit(opts: SecurityAuditOptions): Promise<SecurityAud
  const findings: SecurityAuditFinding[] = [];

  // 第一层：攻击面分析
  findings.push(...collectAttackSurfaceSummaryFindings(cfg));
  findings.push(...collectSyncedFolderFindings({ stateDir, configPath }));

  // 第二层：配置安全检查
  findings.push(...collectGatewayConfigFindings(cfg, env));
  findings.push(...collectBrowserControlFindings(cfg));
  findings.push(...collectLoggingFindings(cfg));
  findings.push(...collectElevatedFindings(cfg));
  findings.push(...collectHooksHardeningFindings(cfg));
  findings.push(...collectSecretsInConfigFindings(cfg));
  findings.push(...collectModelHygieneFindings(cfg));
  findings.push(...collectSmallModelRiskFindings({ cfg, env }));
  findings.push(...collectExposureMatrixFindings(cfg));

  // 第三层：文件系统审计
  if (opts.includeFilesystem !== false) {
    findings.push(...await collectFilesystemFindings({ stateDir, configPath, ... }));
    findings.push(...await collectIncludeFilePermFindings({ configSnapshot, ... }));
    findings.push(...await collectStateDeepFilesystemFindings({ cfg, stateDir, ... }));
    findings.push(...await collectPluginsTrustFindings({ cfg, stateDir }));
    if (opts.deep) {
      findings.push(...await collectPluginsCodeSafetyFindings({ stateDir }));
      findings.push(...await collectInstalledSkillsCodeSafetyFindings({ cfg, stateDir }
    }
  }

  // 第四层：渠道安全检查
  if (opts.includeChannelSecurity !== false) {
    findings.push(...await collectChannelSecurityFindings({ cfg, plugins }));
  }

  // 第五层：深度探测（可选）
  if (opts.deep) {
    const deep = await maybeProbeGateway({ cfg, timeoutMs: 5000, probe });
  }
```

```
  return { ts: Date.now(), summary: countBySeverity(findings), findings, deep };
}
```

Gateway 配置安全检查

`collectGatewayConfigFindings` 检查 Gateway 服务器的安全配置，是最核心的检查项之一：

```typescript
// src/security/audit.ts（简化）

function collectGatewayConfigFindings(cfg, env): SecurityAuditFinding[] {
  const findings = [];

  // 检查 1：非 loopback 绑定但无认证
  if (bind !== "loopback" && !hasSharedSecret) {
    findings.push({
      checkId: "gateway.bind_no_auth",
      severity: "critical",
      title: "Gateway binds beyond loopback without auth",
      detail: `gateway.bind="${bind}" but no auth token/password configured.`,
      remediation: "Set gateway.auth or bind to loopback.",
    });
  }

  // 检查 2：Tailscale Funnel 暴露
  if (tailscaleMode === "funnel") {
    findings.push({
      severity: "critical",
      title: "Tailscale Funnel exposure enabled",
      detail: 'gateway.tailscale.mode="funnel" exposes Gateway publicly.',
    });
  }

  // 检查 3：不安全 HTTP 认证
  if (cfg.gateway?.controlUi?.allowInsecureAuth === true) {
    findings.push({
      severity: "critical",
      title: "Control UI allows insecure HTTP auth",
    });
  }

  // 检查 4：设备认证被禁用
  if (cfg.gateway?.controlUi?.dangerouslyDisableDeviceAuth === true) {
    findings.push({
      severity: "critical",
      title: "DANGEROUS: Control UI device auth disabled",
    });
  }

  // 检查 5：Token 长度不足
  if (token && token.length < 24) {
```

```
    findings.push({
      severity: "warn",
      title: "Gateway token looks short",
      detail: `token is ${token.length} chars; prefer a long random token.`,
    });
  }

  return findings;
}
```

渠道安全检查

渠道安全检查涵盖了 DM 策略、群聊权限、斜杠命令授权等多个维度。以 Discord 为例：

```
// src/security/audit.ts（简化概念）

// 检查 1：斜杠命令无用户限制
if (!useAccessGroups && guildsConfigured && !hasAnyUserAllowlist) {
  findings.push({
    severity: "critical",
    title: "Discord slash commands are unrestricted",
    detail: "Any user in allowed guild channels can invoke /… commands.",
  });
}

// 检查 2：无白名单导致命令全部被拒
if (useAccessGroups && !ownerAllowFromConfigured && !hasAnyUserAllowlist) {
  findings.push({
    severity: "warn",
    title: "Discord slash commands have no allowlists",
    detail: "/… commands will be rejected for everyone.",
  });
}
```

32.4.2 文件系统审计（ src/security/audit-fs.ts ）

权限检查模型

文件系统审计检查 OpenClaw 的关键文件和目录的权限是否安全。 `inspectPathPermissions` 是跨平台的权限检查函数：

```
// src/security/audit-fs.ts（简化）

export async function inspectPathPermissions(targetPath, opts?): Promise<PermissionChec
  const st = await safeStat(targetPath);
  if (!st.ok) return { ok: false, ... };

  const bits = modeBits(st.mode);    // 提取 rwxrwxrwx 位
  const platform = opts?.platform ?? process.platform;

  if (platform === "win32") {
    // Windows：使用 icacls 检查 ACL
    const acl = await inspectWindowsAcl(targetPath, { exec: opts?.exec });
    return { ..., source: "windows-acl", worldWritable: acl.untrustedWorld.some(e => e.
  }

  // POSIX：使用传统的 mode bits
  return {
    ...,
    source: "posix",
    worldWritable: (bits & 0o002) !== 0,   // 其他用户可写?
    groupWritable: (bits & 0o020) !== 0,   // 同组用户可写?
    worldReadable: (bits & 0o004) !== 0,   // 其他用户可读?
    groupReadable: (bits & 0o040) !== 0,   // 同组用户可读?
  };
}
```

> **衍生解释**：Unix/Linux 文件权限使用 9 位二进制数表示，分为三组（所有者/同组/其他），每组 3 位（读/写/执行）。八进制表示法中， `0o700` 意味着所有者有全部权限（rwx），同组和其他用户无任何权限。 `0o600` 意味着所有者可读写，但不可执行。OpenClaw 期望状态目录为 `0o700` （仅所有者可访问），配置文件为 `0o600` （仅所有者可读写），因为配置文件中可能包含 API 密钥等敏感信息。

审计发现的严重级别

文件系统审计根据风险程度分配不同的严重级别：

检查项	条件	严重级别
状态目录世界可写	`bits & 0o002`	`critical`
状态目录组可写	`bits & 0o020`	`warn`
状态目录可被他人读取	`bits & 0o044`	`warn`
配置文件可被他人写入	group/world writable	`critical`
配置文件世界可读	`bits & 0o004`	`critical`
配置文件组可读	`bits & 0o040`	`warn`
路径是符号链接	`isSymbolicLink()`	`warn`

修复建议会根据平台自动生成——POSIX 系统生成 `chmod 700 ~/.openclaw` ，Windows 系统生成 `icacls` 命令。

32.4.3 技能安全扫描（`src/security/skill-scanner.ts`）

为什么需要扫描技能代码

技能（Skills）是用户安装的第三方代码，在 Agent 执行环境中运行。与浏览器扩展类似，恶意技能可能：

- 执行系统命令（`child_process.exec`）
- 窃取环境变量中的 API 密钥
- 通过网络外发敏感数据
- 使用混淆代码隐藏恶意逻辑

扫描规则

技能扫描器使用两类规则——**行级规则（Line Rules）和源码级规则（Source Rules）**：

```
// src/security/skill-scanner.ts（简化）

// 行级规则：逐行匹配
const LINE_RULES = [
  {
    ruleId: "dangerous-exec",
    severity: "critical",
    message: "Shell command execution detected (child_process)",
    pattern: /\b(exec|execSync|spawn|spawnSync)\s*\(/,
    requiresContext: /child_process/,  // 文件中必须也引用了 child_process
  },
  {
    ruleId: "dynamic-code-execution",
    severity: "critical",
    message: "Dynamic code execution detected",
    pattern: /\beval\s*\(|new\s+Function\s*\(/,
  },
  {
    ruleId: "crypto-mining",
    severity: "critical",
    message: "Possible crypto-mining reference detected",
    pattern: /stratum\+tcp|coinhive|cryptonight|xmrig/i,
  },
];

// 源码级规则：跨行匹配
const SOURCE_RULES = [
  {
    ruleId: "potential-exfiltration",
    severity: "warn",
    message: "File read + network send — possible data exfiltration",
    pattern: /readFileSync|readFile/,
    requiresContext: /\bfetch\b|\bpost\b|http\.request/i,
  },
  {
    ruleId: "env-harvesting",
    severity: "critical",
    message: "Environment variable access + network send — possible credential harvesti
    pattern: /process\.env/,
    requiresContext: /\bfetch\b|\bpost\b|http\.request/i,
  },
  {
    ruleId: "obfuscated-code",
```

```
    severity: "warn",
    message: "Large base64 payload with decode call detected",
    pattern: /(?:atob|Buffer\.from)\s*\(\s*["'][A-Za-z0-9+/=]{200,}["']/,
  },
];
```

`requiresContext` 双重检查是一个巧妙的误报控制机制——`exec` 是一个常见的函数名，如果没有 `requiresContext: /child_process/` 的二次验证，正则匹配工具的 `.exec()` 方法也会被误报为危险操作。

目录扫描

`scanDirectory` 遍历技能目录，扫描所有 `.js` / `.ts` / `.mjs` / `.cjs` 等文件：

```
// src/security/skill-scanner.ts（简化）

const SCANNABLE_EXTENSIONS = new Set([".js", ".ts", ".mjs", ".cjs", ".mts", ".cts", ".j
const DEFAULT_MAX_SCAN_FILES = 500;        // 最多扫描 500 个文件
const DEFAULT_MAX_FILE_BYTES = 1024 * 1024; // 单文件最大 1MB

export async function scanDirectoryWithSummary(dirPath, opts?): Promise<SkillScanSummar
  const files = await collectScannableFiles(dirPath, normalizedOpts);
  const allFindings = [];
  let scannedFiles = 0;

  for (const file of files) {
    const source = await readScannableSource(file, maxFileBytes);
    if (source == null) continue;
    scannedFiles++;
    allFindings.push(...scanSource(source, file));
  }

  return { scannedFiles, critical: ..., warn: ..., info: ..., findings: allFindings };
}
```

安全限制保护扫描器本身不被 DoS：

- **文件数上限**（500）：防止巨大的 `node_modules` 目录拖慢扫描
- **文件大小上限**（1MB）：跳过异常大的文件
- **跳过 `node_modules` 和隐藏目录**：避免扫描依赖库

32.4.4 外部内容安全（src/security/external-content.ts）

此模块已在 24.1 节详细分析过。这里补充几个审计相关的 API：

```
// 判断会话是否来自外部钩子
export function isExternalHookSession(sessionKey: string): boolean {
  return sessionKey.startsWith("hook:gmail:") ||
         sessionKey.startsWith("hook:webhook:") ||
         sessionKey.startsWith("hook:");
}

// 提取钩子类型
export function getHookType(sessionKey: string): ExternalContentSource {
  if (sessionKey.startsWith("hook:gmail:")) return "email";
  if (sessionKey.startsWith("hook:webhook:")) return "webhook";
  return "unknown";
}

// Web 内容包装（简化版）
export function wrapWebContent(content: string, source: "web_search" | "web_fetch"): st
  // web_fetch 附带完整安全警告，web_search 仅标记来源
  return wrapExternalContent(content, { source, includeWarning: source === "web_fetch"
}
```

32.4.5 openclaw security audit 命令

安全审计通过 CLI 命令触发：

```
# 基础审计
openclaw security audit

# 深度审计（包含 Gateway 探测和代码安全扫描）
openclaw security audit --deep

# JSON 输出（用于 CI/CD 集成）
openclaw security audit --json
```

审计结果以分级着色的方式输出：

- **CRITICAL**（红色）：需要立即修复的严重安全问题
- **WARN**（黄色）：建议修复的安全隐患
- **INFO**（灰色）：信息性提示

每个发现都附带修复建议（ `remediation` ），例如：

```
CRITICAL: Config file is world-readable
  ~/.openclaw/openclaw.json mode=644; config can contain tokens.
  Fix: chmod 600 ~/.openclaw/openclaw.json
```

本节小结

1. **结构化审计报告**包含三级严重度（critical/warn/info）的发现列表，每个发现附带检查 ID、详情和修复建议。
2. **五层审计流水线**：攻击面分析 → 配置安全检查 → 文件系统权限 → 渠道安全 → 深度探测。
3. **跨平台文件权限检查**：POSIX 使用 mode bits，Windows 使用 icacls ACL，统一为 `worldWritable` / `groupWritable` 等布尔属性。
4. **技能安全扫描器**使用行级+源码级双重规则，配合 `requiresContext` 机制控制误报。
5. **外部内容安全模块**提供会话来源判定和 Web 内容分级包装能力。
6. **CLI 集成**支持基础/深度两种模式和 JSON 输出，适用于手动检查和 CI/CD 自动化。